Assignment 03: Model Optimization Pipeline

Post-Training Quantization, Quantization-Aware Training and Pruning
of ResNet-18 on CIFAR-10

Edge Computing & Intelligent Systems

Hamid Ashraf

Roll No. 22i-0510 Section 7A

Abstract

This report describes a PyTorch pipeline that trains a ResNet-18 baseline on CIFAR-10 and compresses it in four ways: post-training static quantization (PTQ), quantization-aware training (QAT), iterative unstructured pruning and structured channel pruning. All five models are evaluated by one automated benchmark and exported to ONNX for CPU deployment. The FP32 baseline reaches 93.69 % test accuracy. Both INT8 models keep that accuracy at a quarter of the size, 90 %-sparse pruning loses only 0.16 points, and structured pruning halves the parameters and operations while reaching 93.91 %.

1 Setup

Task. Build, compress and compare five ResNet-18 models for CIFAR-10: the FP32 baseline, a PTQ INT8 model, a QAT INT8 model, an unstructured-pruned model and a structured-pruned model, with automated inference, benchmarking and ONNX export.

Environment. NVIDIA GeForce RTX 4060 Laptop GPU (8 GB) for training and fine-tuning, Intel Core i7-14700HX CPU for quantization, INT8 inference and all CPU timing, Windows 11, Python 3.11, PyTorch 2.10 (CUDA build), torchvision 0.25, ONNX 1.23 and ONNX Runtime 1.31. Quantized kernels use the `onednn` backend, which in eager mode runs on the CPU only. No pretrained weights were downloaded; all models are trained from scratch.

Data. The 50 000 CIFAR-10 training images are split into 45 000 for training and 5 000 for validation (fixed seed). The validation set is only used to log progress during training and fine-tuning. The 10 000 test images are used for the final numbers only. Random crop (padding 4) and horizontal flip are applied on the GPU during training.

Model. ResNet-18 adapted to 32×32 inputs: a 3×3 stem convolution without max-pooling, built on torchvision's quantization-ready ResNet so that residual additions can be quantized. It has 11.17 M parameters and 555 M multiply-accumulates (MACs) per image.

Code layout. Table 1 maps every deliverable to its script. Each script runs on its own; `pipeline.ipynb` runs them in order and keeps the outputs shown in the screenshots below.

Deliverable	Script	Output
Baseline model	`train_baseline.py`	`checkpoints/fp32.pth`
PTQ	`ptq.py` (`calibrate()`)	`checkpoints/ptq_int8.pt`
QAT	`qat.py` (`finetune()`)	`checkpoints/qat_int8.pt`
Unstructured pruning	`prune_unstructured.py`	`checkpoints/pruned_unstructured.pth`
Structured pruning	`prune_structured.py`	`checkpoints/pruned_structured.pth`
Inference pipeline	`inference.py`	—
Benchmark	`benchmark.py`	`results/benchmark.csv`, `predictions.csv`
ONNX	`export_onnx.py`, `onnx_inference.py`	`checkpoints/onnx/*.onnx`
Figures	`make_plots.py`	`results/graphs/`

Table 1: Deliverables and the scripts that produce them (`common.py` holds the shared data, model and training code).

In [1]:
```python
import sys
from pathlib import Path
import torch

RETRAIN = False   # True: retrain every stage

def run(script, *args, ckpt=None):
    """Run a script and stream its output; skip a training stage if its checkpoint exists."""
    if ckpt and Path(ckpt).exists() and not RETRAIN:
        print(f"{ckpt} exists - skipped (set RETRAIN = True to retrain)")
    else:
        get_ipython().system(f"{sys.executable} -u {script} {' '.join(args)}")

print("python :", sys.version.split()[0])
print("torch  :", torch.__version__, "| CUDA:", torch.cuda.is_available(), "|", torch.cuda.get_device_
name(0) if torch.cuda.is_available() else "CPU only")
print("quant engines:", torch.backends.quantized.supported_engines)
```

```
python : 3.11.14
torch  : 2.10.0+cu130 | CUDA: True | NVIDIA GeForce RTX 4060 Laptop GPU
quant engines: ['onednn']
```

Figure 1: Notebook setup cell and environment check (GPU, PyTorch version, quantization engine).

2 FP32 Baseline

The baseline is trained for 40 epochs with SGD (momentum 0.9, Nesterov), batch size 256, weight decay 5×10^{-4}, a one-cycle learning-rate schedule with peak 0.1, label smoothing 0.1 and bfloat16 mixed precision. An epoch takes about 11 s on the GPU. The final model is saved as a FP32 checkpoint and reaches **93.69 %** test accuracy (94.40 % on the validation split).

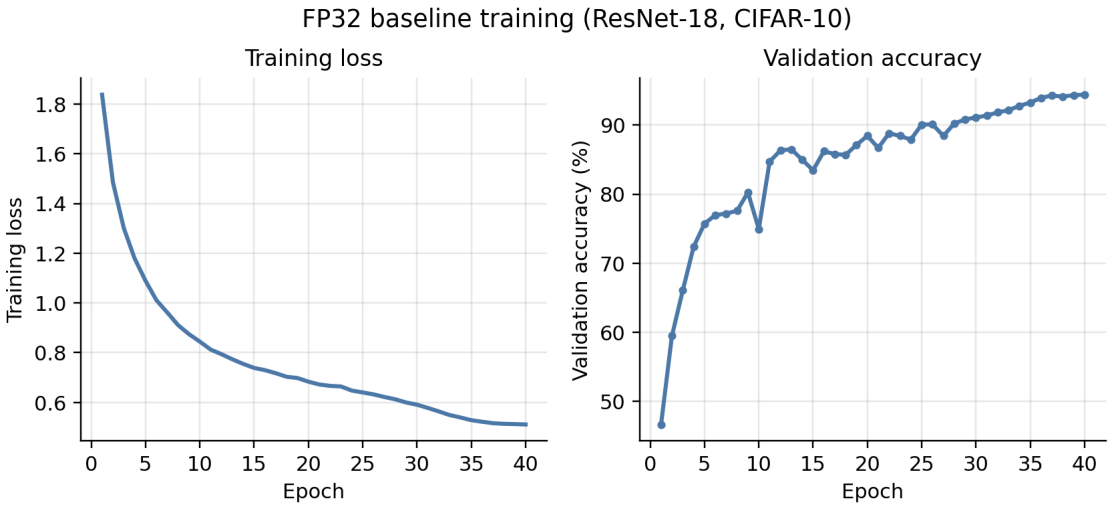


Figure 2: Training loss and validation accuracy of the FP32 baseline.

```
In [2]: run("train_baseline.py", ckpt="checkpoints/fp32.pth")

[fp32] epoch  1/40  loss 1.838  val_acc 46.68%  (12s)
[fp32] epoch  2/40  loss 1.485  val_acc 59.50%  (11s)
[fp32] epoch  3/40  loss 1.301  val_acc 66.08%  (11s)
[fp32] epoch  4/40  loss 1.179  val_acc 72.40%  (11s)
[fp32] epoch  5/40  loss 1.089  val_acc 75.74%  (11s)
[fp32] epoch  6/40  loss 1.011  val_acc 76.98%  (11s)
[fp32] epoch  7/40  loss 0.962  val_acc 77.20%  (11s)
[fp32] epoch  8/40  loss 0.911  val_acc 77.60%  (11s)
[fp32] epoch  9/40  loss 0.875  val_acc 80.26%  (11s)
[fp32] epoch 10/40  loss 0.845  val_acc 74.96%  (11s)
[fp32] epoch 11/40  loss 0.813  val_acc 84.72%  (11s)
[fp32] epoch 12/40  loss 0.794  val_acc 86.34%  (11s)
[fp32] epoch 13/40  loss 0.773  val_acc 86.50%  (11s)
[fp32] epoch 14/40  loss 0.755  val_acc 84.98%  (11s)
[fp32] epoch 15/40  loss 0.739  val_acc 83.46%  (10s)
[fp32] epoch 16/40  loss 0.730  val_acc 86.22%  (11s)
[fp32] epoch 17/40  loss 0.717  val_acc 85.80%  (11s)
[fp32] epoch 18/40  loss 0.703  val_acc 85.70%  (11s)
[fp32] epoch 19/40  loss 0.698  val_acc 87.14%  (11s)
[fp32] epoch 20/40  loss 0.683  val_acc 88.44%  (11s)
[fp32] epoch 21/40  loss 0.672  val_acc 86.70%  (11s)
[fp32] epoch 22/40  loss 0.667  val_acc 88.82%  (11s)
[fp32] epoch 23/40  loss 0.664  val_acc 88.44%  (10s)
[fp32] epoch 24/40  loss 0.648  val_acc 87.90%  (10s)
[fp32] epoch 25/40  loss 0.640  val_acc 90.08%  (10s)
[fp32] epoch 26/40  loss 0.632  val_acc 90.10%  (10s)
[fp32] epoch 27/40  loss 0.622  val_acc 88.44%  (10s)
[fp32] epoch 28/40  loss 0.613  val_acc 90.22%  (10s)
[fp32] epoch 29/40  loss 0.600  val_acc 90.82%  (10s)
[fp32] epoch 30/40  loss 0.591  val_acc 91.10%  (11s)
[fp32] epoch 31/40  loss 0.578  val_acc 91.38%  (11s)
[fp32] epoch 32/40  loss 0.564  val_acc 91.84%  (10s)
[fp32] epoch 33/40  loss 0.549  val_acc 92.16%  (11s)
[fp32] epoch 34/40  loss 0.539  val_acc 92.78%  (11s)
[fp32] epoch 35/40  loss 0.529  val_acc 93.26%  (11s)
[fp32] epoch 36/40  loss 0.522  val_acc 93.94%  (11s)
[fp32] epoch 37/40  loss 0.516  val_acc 94.28%  (11s)
[fp32] epoch 38/40  loss 0.514  val_acc 94.12%  (11s)
[fp32] epoch 39/40  loss 0.513  val_acc 94.34%  (11s)
[fp32] epoch 40/40  loss 0.511  val_acc 94.40%  (11s)

FP32 baseline test accuracy: 93.69%  -> checkpoints/fp32.pth
```

Figure 3: Notebook output of the baseline training stage.

3 Post-Training Static Quantization

PTQ converts the trained FP32 model to INT8 without further training (`ptq.py`):

1. **Fusion.** Each Conv–BN–ReLU group (and Conv–BN in the residual branches) is fused into one module, so the batch-norm statistics are folded into the convolution weights.
2. **Backend.** The quantization engine is set to `onednn` and the default qconfig of that backend is attached to the model.
3. **Calibration.** Observers are inserted and 2 000 randomly chosen training images (batches of 100, no augmentation) are passed through the model so each observer records the activation range from which the INT8 scale and zero-point are derived.
4. **Conversion.** The observed model is converted to a real INT8 model.
5. **Deployment.** The model is saved as a TorchScript file, which loads and runs without the model class.

The PTQ model reaches **93.70 %** test accuracy, the same as FP32, and its checkpoint is 10.89 MB instead of 42.69 MB (3.9× smaller). It agrees with the FP32 model on 99.73 % of the test predictions.

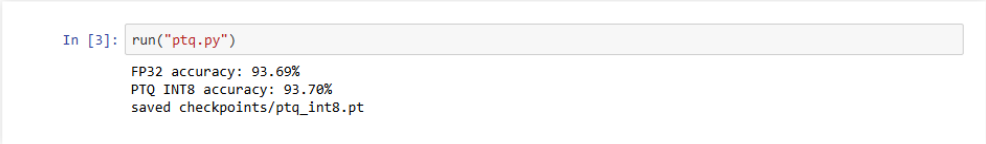


Figure 4: Notebook output of the PTQ stage.

4 Quantization-Aware Training

QAT (`qat.py`) starts from the pretrained FP32 weights, fuses the layers in training mode, attaches the QAT qconfig and calls `prepare_qat`, which inserts fake-quantization modules. The model is then fine-tuned for 8 epochs on the GPU in FP32 arithmetic with SGD and a one-cycle schedule (peak learning rate 0.005), so the weights adapt to the rounding error of INT8. Quantizer observers are frozen from the sixth epoch and the batch-norm statistics from the seventh, so the last epochs train against fixed quantization parameters. After training the model is moved to the CPU, converted to a real INT8 model and exported as a TorchScript file.

The QAT model reaches **93.69 %** test accuracy with the same 10.89 MB checkpoint as PTQ. For this network PTQ is already almost lossless, so QAT cannot improve on it; it has more value for models that PTQ degrades. The QAT model agrees with FP32 on 98.28 % of test predictions, lower than PTQ because fine-tuning changes the weights, so it is a different model rather than a rounded copy of the baseline.

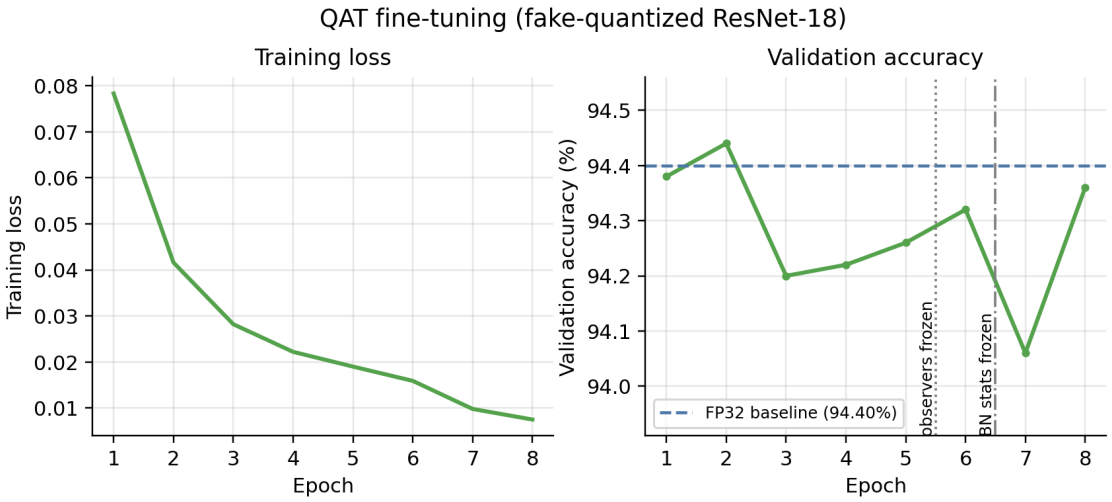


Figure 5: QAT fine-tuning. The validation axis spans only 0.4 points, so the variation between epochs is small in absolute terms.

```
In [4]: run("qat.py", ckpt="checkpoints/qat_int8.pt")

        [qat] epoch  1/8  loss 0.078  val_acc 94.38%  (64s)
        [qat] epoch  2/8  loss 0.042  val_acc 94.44%  (65s)
        [qat] epoch  3/8  loss 0.028  val_acc 94.20%  (64s)
        [qat] epoch  4/8  loss 0.022  val_acc 94.22%  (64s)
        [qat] epoch  5/8  loss 0.019  val_acc 94.26%  (63s)
        [qat] epoch  6/8  loss 0.016  val_acc 94.32%  (63s)
        [qat] epoch  7/8  loss 0.010  val_acc 94.06%  (37s)
        [qat] epoch  8/8  loss 0.007  val_acc 94.36%  (38s)

        QAT INT8 accuracy: 93.69%
        saved checkpoints/qat_int8.pt
```

Figure 6: Notebook output of the QAT stage.

5 Unstructured Pruning

`prune_unstructured.py` applies global magnitude (L1) pruning to the weights of all convolutional and fully connected layers using `torch.nn.utils.prune`. Pruning is iterative: five iterations raise the sparsity to 40, 60, 75, 85 and 90 %, and each iteration is followed by 6 epochs of fine-tuning (learning rate 0.02) with the masks active, so removed weights stay zero. Each step prunes a fraction of the weights that remain, computed from the cumulative target. After the last iteration `prune.remove` makes the pruning permanent by multiplying each weight by its mask and discarding the mask.

Iteration	Sparsity (%)	Val. acc. after pruning (%)	Val. acc. after fine-tuning (%)
0	0	94.40	94.40
1	40	94.30	94.10
2	60	93.20	93.94
3	75	92.68	93.76
4	85	83.90	93.84
5	90	70.26	93.68

Table 2: Iterative unstructured pruning (validation split).

The convolutional and fully connected weights of the final model are 90.0 % zero; counting the

batch-norm and bias parameters as well, 1 126 045 of 11 173 962 parameters remain non-zero. It reaches **93.53 %** test accuracy, 0.16 points below the baseline. Pruning alone drops the accuracy sharply at 85–90 % sparsity (Figure 7), and fine-tuning recovers almost all of it. The pruned weights are stored as zeros in a dense tensor, so the checkpoint is as large as the FP32 one (42.70 MB), but it compresses to 7.69 MB with gzip against 39.60 MB for the dense model. Dense convolution kernels do not skip zeros, so the saving is in storage with a sparse format, not in inference speed (Section 8).

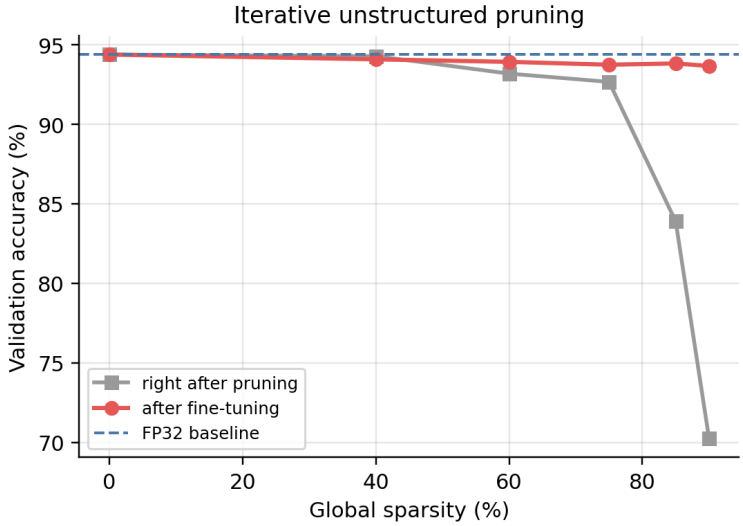


Figure 7: Accuracy against global sparsity, right after each pruning step and after fine-tuning.

```
In [5]: run("prune_unstructured.py", ckpt="checkpoints/pruned_unstructured.pth")

[prune 1/5 @ 40%] epoch  1/6  loss 0.058  val_acc 93.46%  (11s)
[prune 1/5 @ 40%] epoch  2/6  loss 0.034  val_acc 92.38%  (11s)
[prune 1/5 @ 40%] epoch  3/6  loss 0.039  val_acc 92.96%  (11s)
[prune 1/5 @ 40%] epoch  4/6  loss 0.036  val_acc 92.98%  (11s)
[prune 1/5 @ 40%] epoch  5/6  loss 0.022  val_acc 94.06%  (11s)
[prune 1/5 @ 40%] epoch  6/6  loss 0.013  val_acc 94.10%  (11s)
[prune 2/5 @ 60%] epoch  1/6  loss 0.017  val_acc 93.18%  (11s)
[prune 2/5 @ 60%] epoch  2/6  loss 0.032  val_acc 92.84%  (11s)
[prune 2/5 @ 60%] epoch  3/6  loss 0.037  val_acc 91.36%  (11s)
[prune 2/5 @ 60%] epoch  4/6  loss 0.026  val_acc 93.58%  (11s)
[prune 2/5 @ 60%] epoch  5/6  loss 0.017  val_acc 93.88%  (11s)
[prune 2/5 @ 60%] epoch  6/6  loss 0.011  val_acc 93.94%  (11s)
[prune 3/5 @ 75%] epoch  1/6  loss 0.021  val_acc 93.08%  (11s)
[prune 3/5 @ 75%] epoch  2/6  loss 0.028  val_acc 92.32%  (11s)
[prune 3/5 @ 75%] epoch  3/6  loss 0.028  val_acc 92.72%  (11s)
[prune 3/5 @ 75%] epoch  4/6  loss 0.025  val_acc 92.92%  (11s)
[prune 3/5 @ 75%] epoch  5/6  loss 0.015  val_acc 93.66%  (11s)
[prune 3/5 @ 75%] epoch  6/6  loss 0.009  val_acc 93.76%  (11s)
[prune 4/5 @ 85%] epoch  1/6  loss 0.034  val_acc 93.00%  (11s)
[prune 4/5 @ 85%] epoch  2/6  loss 0.027  val_acc 92.88%  (11s)
[prune 4/5 @ 85%] epoch  3/6  loss 0.027  val_acc 92.68%  (11s)
[prune 4/5 @ 85%] epoch  4/6  loss 0.019  val_acc 93.04%  (11s)
[prune 4/5 @ 85%] epoch  5/6  loss 0.014  val_acc 93.72%  (11s)
[prune 4/5 @ 85%] epoch  6/6  loss 0.008  val_acc 93.84%  (11s)
[prune 5/5 @ 90%] epoch  1/6  loss 0.053  val_acc 92.46%  (11s)
[prune 5/5 @ 90%] epoch  2/6  loss 0.032  val_acc 91.78%  (11s)
[prune 5/5 @ 90%] epoch  3/6  loss 0.028  val_acc 92.54%  (11s)
[prune 5/5 @ 90%] epoch  4/6  loss 0.022  val_acc 93.06%  (11s)
[prune 5/5 @ 90%] epoch  5/6  loss 0.013  val_acc 93.56%  (11s)
[prune 5/5 @ 90%] epoch  6/6  loss 0.009  val_acc 93.68%  (11s)
 iteration  target_sparsity  sparsity  val_acc_after_prune  val_acc_after_finetune
         0              0.0       0.0                94.40                   94.40
         1             40.0      40.0                94.30                   94.10
         2             60.0      60.0                93.20                   93.94
         3             75.0      75.0                92.68                   93.76
         4             85.0      85.0                83.90                   93.84
         5             90.0      90.0                70.26                   93.68

Final: 90.0% sparse, 1,126,045/11,173,962 non-zero params, test acc 93.53%  -> checkpoints/pruned_unst
ructured.pth
```

Figure 8: Notebook output of the unstructured pruning stage.

6 Structured Channel Pruning

Structured pruning removes whole channels, so the pruned network is physically smaller. In each of the eight BasicBlocks the path is $x \to$ conv1 $\to$ bn1 $\to$ ReLU $\to$ conv2 $\to$ bn2, added to the skip connection. `prune_structured.py` ranks the output filters of conv1 by their L1 norm and keeps the 50 % with the largest norm. Removing a channel removes one filter of conv1, one entry of bn1 (weight, bias, running mean and variance) and the matching input slice of conv2, so the tensor dimensions of the three layers stay consistent. The input and output widths of the block do not change, so the residual additions stay valid. A new network is built with the reduced widths and the surviving weights are copied into it. It contains no masked zero channels, and its configuration (per-block widths) is stored in the checkpoint so it can be rebuilt when loading.

Block	B1	B2	B3	B4	B5	B6	B7	B8
Original inner channels	64	64	128	128	256	256	512	512
Pruned inner channels	32	32	64	64	128	128	256	256

Table 3: Inner channels (conv1 output / conv2 input) of each BasicBlock before and after pruning.

The pruned model has 5.68 M parameters instead of 11.17 M (49 % fewer) and 282 M MACs instead of 555 M (49 % fewer). Immediately after pruning the validation accuracy falls to 39.30 %; 30 epochs of fine-tuning (peak learning rate 0.05, label smoothing 0.1) restore it to **93.91 %** test accuracy, slightly above the baseline. The checkpoint is 21.73 MB. Only the inner channels of the blocks are pruned. The stem and the channels that run along the residual connections are kept, because pruning them would require removing the same channels in every layer that adds into them.

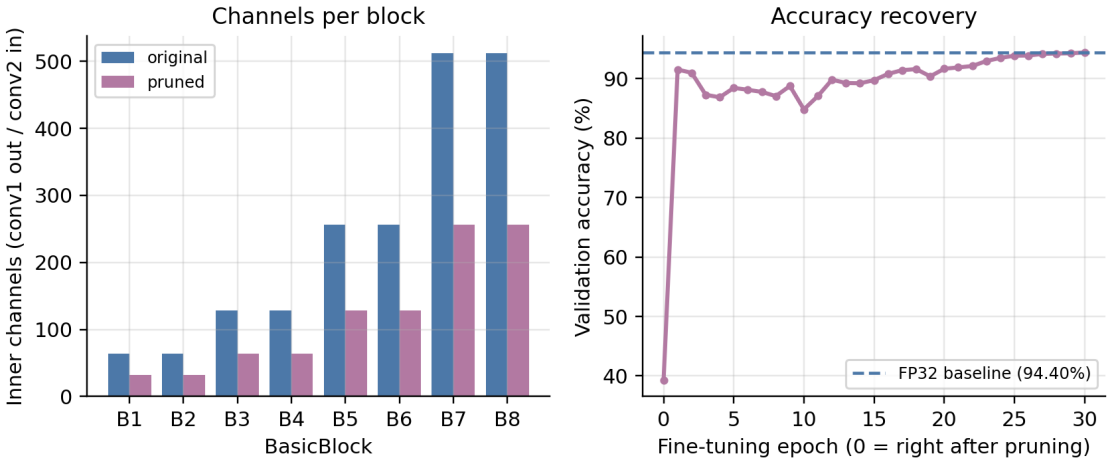


Figure 9: Channel widths per block (left) and accuracy recovery during fine-tuning (right).

```
In [6]: run("prune_structured.py", ckpt="checkpoints/pruned_structured.pth")

inner widths per block: [32, 32, 64, 64, 128, 128, 256, 256]
params 11.17M -> 5.68M (49% fewer), MACs 555M -> 282M (49% fewer)
validation accuracy right after pruning (no fine-tune): 39.30%
[structured] epoch  1/30  loss 0.599  val_acc 91.54%  (7s)
[structured] epoch  2/30  loss 0.578  val_acc 91.00%  (7s)
[structured] epoch  3/30  loss 0.613  val_acc 87.30%  (7s)
[structured] epoch  4/30  loss 0.639  val_acc 86.88%  (7s)
[structured] epoch  5/30  loss 0.648  val_acc 88.46%  (7s)
[structured] epoch  6/30  loss 0.648  val_acc 88.14%  (7s)
[structured] epoch  7/30  loss 0.644  val_acc 87.78%  (7s)
[structured] epoch  8/30  loss 0.634  val_acc 87.08%  (7s)
[structured] epoch  9/30  loss 0.628  val_acc 88.78%  (7s)
[structured] epoch 10/30  loss 0.625  val_acc 84.84%  (7s)
[structured] epoch 11/30  loss 0.614  val_acc 87.10%  (7s)
[structured] epoch 12/30  loss 0.612  val_acc 89.84%  (7s)
[structured] epoch 13/30  loss 0.606  val_acc 89.28%  (7s)
[structured] epoch 14/30  loss 0.596  val_acc 89.24%  (7s)
[structured] epoch 15/30  loss 0.595  val_acc 89.76%  (7s)
[structured] epoch 16/30  loss 0.585  val_acc 90.82%  (7s)
[structured] epoch 17/30  loss 0.583  val_acc 91.44%  (7s)
[structured] epoch 18/30  loss 0.576  val_acc 91.62%  (7s)
[structured] epoch 19/30  loss 0.567  val_acc 90.40%  (7s)
[structured] epoch 20/30  loss 0.560  val_acc 91.68%  (7s)
[structured] epoch 21/30  loss 0.551  val_acc 91.90%  (7s)
[structured] epoch 22/30  loss 0.542  val_acc 92.14%  (7s)
[structured] epoch 23/30  loss 0.536  val_acc 92.98%  (7s)
[structured] epoch 24/30  loss 0.526  val_acc 93.50%  (7s)
[structured] epoch 25/30  loss 0.519  val_acc 93.88%  (7s)
[structured] epoch 26/30  loss 0.514  val_acc 93.90%  (7s)
[structured] epoch 27/30  loss 0.510  val_acc 94.18%  (7s)
[structured] epoch 28/30  loss 0.508  val_acc 94.24%  (7s)
[structured] epoch 29/30  loss 0.507  val_acc 94.28%  (7s)
[structured] epoch 30/30  loss 0.507  val_acc 94.42%  (7s)

Structured-pruned test accuracy: 93.91%  -> checkpoints/pruned_structured.pth
```

Figure 10: Notebook output of the structured pruning stage.

7 Inference Pipeline

`inference.py` loads any of the five models by name (`fp32`, `ptq_int8`, `qat_int8`, `unstructured`, `structured`) through one function and classifies either CIFAR-10 test images or image files supplied by the user (resized to 32×32). It prints the predicted class, the confidence, and, for test images, whether the prediction is correct. INT8 models are TorchScript files and always run on the CPU; the other models run on the CPU or GPU.

```
In [7]: for m in ["fp32", "ptq_int8", "qat_int8", "unstructured", "structured"]:
            run("inference.py", "--model", m, "--n", "5")
            print()

model: fp32 on cpu
     test[0]: cat        (90.7%)  true=cat        OK
     test[1]: ship       (91.7%)  true=ship       OK
     test[2]: ship       (84.3%)  true=ship       OK
     test[3]: airplane   (93.3%)  true=airplane   OK
     test[4]: frog       (91.6%)  true=frog       OK


model: ptq_int8 on cpu
     test[0]: cat        (90.7%)  true=cat        OK
     test[1]: ship       (91.7%)  true=ship       OK
     test[2]: ship       (83.6%)  true=ship       OK
     test[3]: airplane   (93.3%)  true=airplane   OK
     test[4]: frog       (91.6%)  true=frog       OK


model: qat_int8 on cpu
     test[0]: cat        (99.7%)  true=cat        OK
     test[1]: ship       (99.7%)  true=ship       OK
     test[2]: ship       (96.2%)  true=ship       OK
     test[3]: airplane   (99.7%)  true=airplane   OK
     test[4]: frog       (99.7%)  true=frog       OK


model: unstructured on cpu
     test[0]: cat        (100.0%)  true=cat        OK
     test[1]: ship       (99.8%)  true=ship       OK
     test[2]: ship       (98.3%)  true=ship       OK
     test[3]: airplane   (99.9%)  true=airplane   OK
     test[4]: frog       (99.9%)  true=frog       OK


model: structured on cpu
     test[0]: cat        (90.4%)  true=cat        OK
     test[1]: ship       (89.9%)  true=ship       OK
     test[2]: automobile (80.8%)  true=ship       WRONG
     test[3]: airplane   (92.1%)  true=airplane   OK
     test[4]: frog       (91.0%)  true=frog       OK
```

Figure 11: Inference pipeline output for the five models on the first five test images.

8 Benchmarking

`benchmark.py` evaluates the five models on the 10 000 CIFAR-10 test images, all on the CPU so that FP32 and INT8 are compared under the same conditions. It records the accuracy, checkpoint size (plain and gzip-compressed), parameters, non-zero parameters, MACs, CPU latency for a single image and CPU throughput for a batch of 128. Latency and throughput are measured with 4 threads after a short CPU warm-up: each is the best of three rounds, where a round is the median of 200 single-image calls (latency) or 20 batch calls (throughput) after untimed warm-up calls. The per-image predictions of every model are written to `results/predictions.csv` and the metrics to `results/benchmark.csv`.

	FP32	PTQ INT8	QAT INT8	Unstruct.	Struct.
Test accuracy (%)	93.69	93.70	93.69	93.53	93.91
Checkpoint size (MB)	42.69	10.89	10.89	42.70	21.73
Size after gzip (MB)	39.60	9.69	9.69	7.69	20.13
Parameters (M)	11.17	11.17	11.17	11.17	5.68
Non-zero parameters (M)	11.17	11.17	11.17	1.13	5.68
MACs (M)	555	555	555	555	282
CPU latency, batch 1 (ms)	10.42	22.73	35.38	10.47	3.12
CPU throughput, batch 128 (img/s)	317	1543	421	151	269

Table 4: Benchmark of the five models on the CIFAR-10 test set (CPU, 4 threads). For the INT8 models the parameter and MAC counts are those of the FP32 architecture.

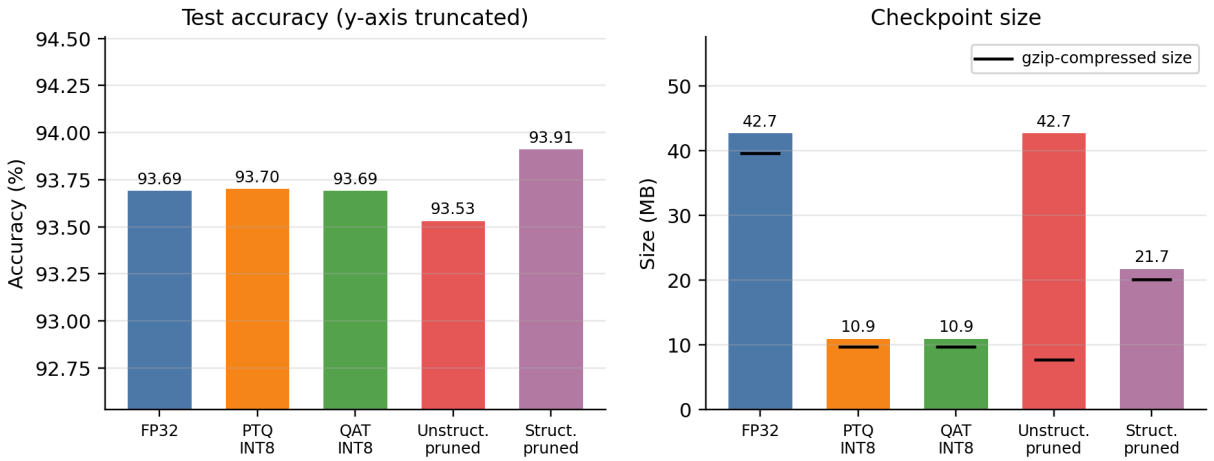


Figure 12: Test accuracy (the y-axis is truncated) and checkpoint size; the black marks show the gzip-compressed size.

Accuracy. All five models lie between 93.53 % and 93.91 %. With 10 000 test images the standard error of an accuracy near 94 % is about 0.24 points, so these differences are within the noise of a single evaluation, and the fair conclusion is that compression did not cost measurable accuracy. This is the result of one training run with one seed.

Size and operations. INT8 reduces the checkpoint 3.9× (42.69 to 10.89 MB). Structured pruning halves the parameters, the MACs and the file (21.73 MB). Unstructured pruning leaves the dense file unchanged but removes 90 % of the weights, which shows in the gzip size (7.69 MB, the smallest of all models) and in the non-zero parameter count.

Speed and its variability. The PyTorch CPU timings in Table 4 are not reliable on this machine. The laptop CPU mixes performance and efficiency cores and throttles with heat, and

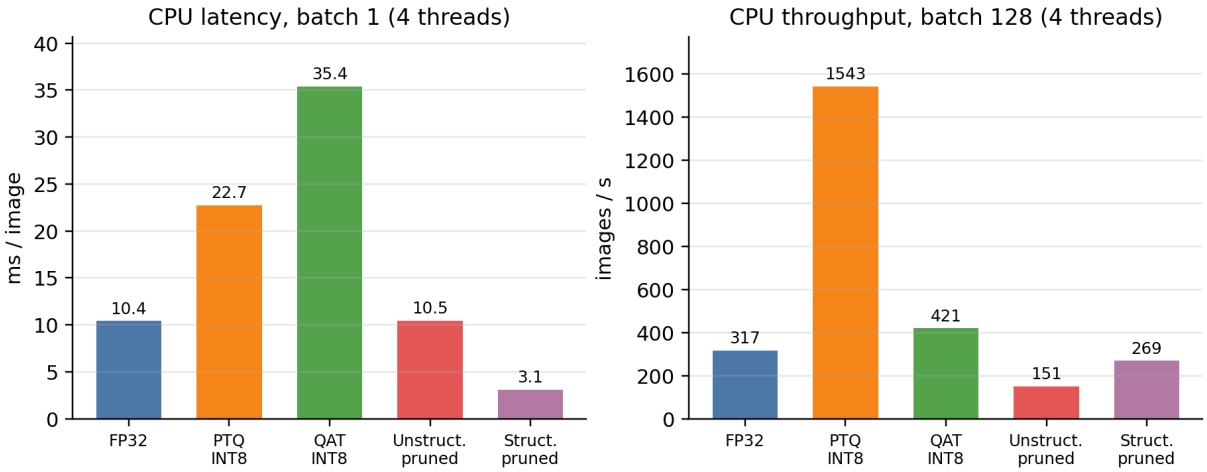


Figure 13: PyTorch CPU latency and throughput. These timings vary between runs on this laptop (see the discussion below).

repeated runs of the same benchmark gave latencies that differed by up to a factor of two to three for the same model; for instance PTQ and QAT, which have identical architectures and operations, are listed at 22.73 and 35.38 ms. Differences of this kind come from the measurement environment, not from the models. Eager-mode PyTorch also pays a fixed overhead per INT8 operator call, which hurts single-image latency. The PyTorch timings should therefore be read as indicative only, and the ONNX Runtime measurements in Section 9, which were consistent across repeated runs, are the better guide to deployment speed. GPU latency for a single image (also recorded in the CSV) is dominated by kernel-launch overhead and is not interpreted here.

```
In [9]: import pandas as pd
        bench = pd.read_csv("results/benchmark.csv")
        bench.set_index("model").T   # one column per model
```

Out[9]:

model	fp32	ptq_int8	qat_int8	unstructured	structured
accuracy_pct	93.69	93.70	93.69	93.53	93.91
file_size_mb	42.69	10.89	10.89	42.70	21.73
gzip_size_mb	39.60	9.69	9.69	7.69	20.13
params_m	11.17	11.17	11.17	11.17	5.68
nonzero_params_m	11.17	11.17	11.17	1.13	5.68
macs_m	555.00	555.00	555.00	555.00	282.00
cpu_latency_ms_bs1	10.42	22.73	35.38	10.47	3.12
cpu_throughput_img_s_bs128	317.00	1543.00	421.00	151.00	269.00
gpu_latency_ms_bs1	1.80	NaN	NaN	4.56	1.94

Figure 14: Contents of `results/benchmark.csv` as shown in the notebook (one column per model).

```
In [10]: pd.read_csv("results/predictions.csv").head(10)
Out[10]:
```

	index	label	fp32	ptq_int8	qat_int8	unstructured	structured
0	0	3	3	3	3	3	3
1	1	8	8	8	8	8	8
2	2	8	8	8	8	8	1
3	3	0	0	0	0	0	0
4	4	6	6	6	6	6	6
5	5	6	6	6	6	6	6
6	6	1	1	1	1	1	1
7	7	6	6	6	6	6	6
8	8	3	3	3	3	3	3
9	9	1	1	1	1	1	1

Figure 15: First rows of `results/predictions.csv`: the true label and the class predicted by each model for every test image.

9 ONNX Deployment

`export_onnx.py` exports the FP32, unstructured-pruned and structured-pruned models to ONNX (opset 17, dynamic batch dimension). PyTorch's eager-mode INT8 models cannot be exported to ONNX, so an INT8 ONNX model is produced on the ONNX side: ONNX Runtime's static quantizer converts the FP32 ONNX model to INT8 (QDQ format, uint8 activations, int8 weights) using 1 000 training images for calibration. This model is quantized from the FP32 network by a different tool, so it is a separate model from the PyTorch PTQ and QAT checkpoints. `onnx_inference.py` runs every ONNX file with ONNX Runtime's CPU provider (4 threads), reports the test accuracy and measures latency and throughput the same way as above.

	FP32	FP32 INT8 (ORT)	Unstruct.	Struct.
Test accuracy (%)	93.69	93.79	93.53	93.91
File size (MB)	42.62	10.71	42.62	21.66
Latency, batch 1 (ms)	2.77	0.90	2.47	1.33
Throughput, batch 128 (img/s)	449	1480	465	872

Table 5: ONNX Runtime CPU results (4 threads).

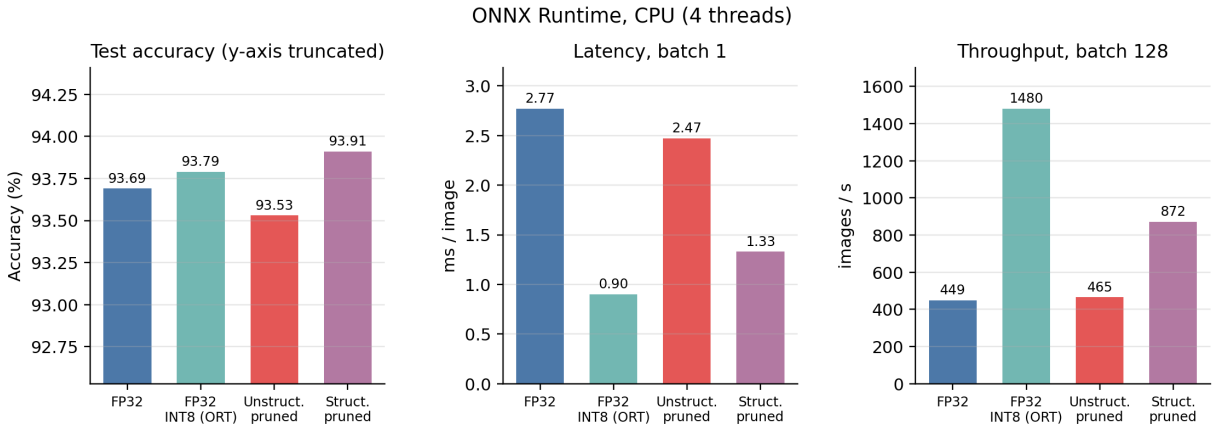


Figure 16: ONNX Runtime accuracy, latency and throughput.

With ONNX Runtime the speed differences follow the amount of computation. The INT8 model

is 3.1× faster per image (0.90 against 2.77 ms) and 3.3× higher in throughput than FP32, with the same accuracy and a quarter of the size. The structured model is 2.1× faster per image and 1.9× higher in throughput, consistent with its halved MACs. The unstructured model is only about 1.1× faster, as expected, because the dense kernels still process every zero.

```
In [11]: run("export_onnx.py")

exported fp32.onnx
exported unstructured.onnx
exported structured.onnx
exported fp32_int8.onnx (ONNX Runtime static INT8)

In [12]: run("onnx_inference.py")

fp32.onnx          acc 93.69%
fp32_int8.onnx     acc 93.79%
structured.onnx    acc 93.91%
unstructured.onnx  acc 93.53%

(ONNX Runtime CPU, 4 threads)
       onnx_model  accuracy_pct  file_size_mb  cpu_latency_ms_bs1  cpu_throughput_img_s_bs128
        fp32.onnx         93.69         42.62                2.77                         449
     fp32_int8.onnx         93.79         10.71                0.90                        1480
     structured.onnx         93.91         21.66                1.33                         872
   unstructured.onnx         93.53         42.62                2.47                         465
```

Figure 17: Notebook output of the ONNX export and ONNX Runtime benchmark.

10 Conclusions

- All five models stay within about 0.2 points of the 93.69 % baseline on the test set, a difference smaller than the evaluation noise.
- **INT8 quantization** gives a 3.9× smaller checkpoint, and with ONNX Runtime about 3× lower latency, at no measurable accuracy cost. PTQ was enough for this model; QAT matched it.
- **Structured pruning** is the option that reduces real work: half the parameters and MACs, a half-size model and about 2× faster ONNX inference, without special sparse kernels.
- **Unstructured pruning** reaches 90 % sparsity almost for free in accuracy and gives the smallest compressed file, but dense hardware gets no speedup from it; it needs a sparse runtime to pay off.

Limitations. The results come from one training run per model. PyTorch CPU timings were noisy on the laptop used, so speed conclusions rest on the ONNX Runtime numbers. Structured pruning covers only the inner channels of each block. Combining the techniques, for example quantizing the structured-pruned model, was not part of this assignment.

Reproducing. With CIFAR-10 unpacked in `data/cifar-10-batches-py`, run the scripts in the order given in the `README.md` (`train_baseline.py`, `ptq.py`, `qat.py`, `prune_unstructured.py`, `prune_structured.py`, `benchmark.py`, `export_onnx.py`, `onnx_inference.py`), or run all cells of `pipeline.ipynb`. Seeds are fixed, and a full run takes about 30 minutes on the GPU used here.